

MERN Full Stack Final Assessment (FA)

SET B — Issue / Bug Tracking System

API Response & Frontend Evaluator Contract

1. Purpose of This Document

This document defines the:

- expected API response formats
- status codes
- pagination structures
- authentication response structures
- analytics response formats
- frontend evaluator conventions
- Playwright compatibility requirements

for the:

Issue / Bug Tracking System — Set B

Students must strictly follow:

- route names
- response structures
- status codes
- data-testid conventions
- window.appState structure

mentioned in this document.

Failure to follow these conventions may affect automated evaluation.

2. Common Backend Response Standards

Success Response

Status Code

200 / 201

Expected Structure

```
{
  "success": true,
  "message": "Operation successful",
  "data": {}
}
```

Error Response

Status Codes

Scenario	Status
Validation Error	400
Unauthorized	401
Forbidden	403
Not Found	404
Server Error	500

Expected Structure

```
{
  "success": false,
  "message": "Error message"
}
```

Pagination Response Format

Expected Structure

```
{
  "success": true,
  "message": "Data fetched successfully",
  "page": 1,
  "limit": 10,
  "total": 50,
  "totalPages": 5,
  "data": [
    {
      "_id": "string"
    }
  ]
}
```

3. MongoDB & Authentication Routes

POST /sync

Status Code

200

Auth Required

Yes

Allowed Roles

admin

manager

Expected Response

```
{
  "success": true,
  "message": "Dataset synchronized successfully",
  "data": {
    "totalFetched": 100,
    "inserted": 84,
    "duplicates": 6,
    "rejected": 10
  }
}
```

Values are dynamic based on dataset.

GET /health**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Database connected successfully",
  "data": {
    "database": "connected",
    "documentCount": 84
  }
}
```

POST /auth/register**Status Code**

201

Expected Response

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "_id": "string",
    "userId": "string",
    "name": "string",
  }
}
```

```
{
  "email": "string",
  "role": "developer",
  "createdAt": "ISO_DATE"
}
```

POST /auth/login

Status Code

200

Expected Response

```
{
  "success": true,
  "message": "Login successful",
  "token": "JWT_TOKEN",
  "data": {
    "_id": "string",
    "userId": "string",
    "name": "string",
    "email": "string",
    "role": "manager"
  }
}
```

GET /auth/me

Status Code

200

Auth Required

Yes

Expected Response

```
{
  "success": true,
  "message": "Authenticated user fetched successfully",
  "data": {
    "_id": "string",
    "userId": "string",
    "name": "string",
    "email": "string",
    "role": "developer"
  }
}
```

4. User APIs

GET /users

Status Code

200

Pagination Supported

Yes

Expected Response

```
{
  "success": true,
  "message": "Users fetched successfully",
  "page": 1,
  "limit": 10,
  "total": 50,
  "totalPages": 5,
  "data": [
    {
      "_id": "string",
      "userId": "USR101",
      "name": "Rahul",
      "email": "rahul@test.com",
      "role": "developer",
      "status": "active"
    }
  ]
}
```

GET /users/:id

Status Code

200

Expected Response

```
{
  "success": true,
  "message": "User fetched successfully",
  "data": {
    "_id": "string",
    "userId": "USR101",
    "name": "Rahul",
    "email": "rahul@test.com",
    "role": "developer",
    "department": "Frontend",
    "status": "active"
  }
}
```

```
}  
}
```

5. Project APIs

POST /projects

Status Code

201

Expected Response

```
{  
  "success": true,  
  "message": "Project created successfully",  
  "data": {  
    "_id": "string",  
    "projectId": "PROJ1001",  
    "title": "Bug Tracker",  
    "status": "active",  
    "createdAt": "ISO_DATE"  
  }  
}
```

GET /projects

Status Code

200

Expected Response

```
{  
  "success": true,  
  "message": "Projects fetched successfully",  
  "data": [  
    {  
      "_id": "string",  
      "projectId": "PROJ1001",  
      "title": "Bug Tracker",  
      "status": "active"  
    }  
  ]  
}
```

GET /projects/:id**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Project fetched successfully",
  "data": {
    "_id": "string",
    "projectId": "PROJ1001",
    "title": "Bug Tracker",
    "description": "Issue management system",
    "owner": {},
    "members": [],
    "status": "active"
  }
}
```

PATCH /projects/:id**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Project updated successfully",
  "data": {
    "_id": "string",
    "projectId": "PROJ1001",
    "status": "active",
    "updatedAt": "ISO_DATE"
  }
}
```

DELETE /projects/:id**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Project deleted successfully"
}
```

```
}
```

6. Issue APIs

POST /issues

Status Code

201

Expected Response

```
{
  "success": true,
  "message": "Issue created successfully",
  "data": {
    "_id": "string",
    "issueId": "ISS101",
    "title": "Login page crash",
    "priority": "high",
    "severity": "critical",
    "status": "open",
    "createdAt": "ISO_DATE"
  }
}
```

GET /issues

Status Code

200

Pagination Supported

Yes

Expected Response

```
{
  "success": true,
  "message": "Issues fetched successfully",
  "page": 1,
  "limit": 10,
  "total": 50,
  "totalPages": 5,
  "data": [
    {
      "_id": "string",
      "issueId": "ISS101",
      "title": "Login page crash",
      "priority": "high",
```

```
    "status": "open"
  }
]
}
```

GET /issues/:id**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Issue fetched successfully",
  "data": {
    "_id": "string",
    "issueId": "ISS101",
    "title": "Login page crash",
    "description": "App crashes after login",
    "project": {},
    "assignedTo": {},
    "reportedBy": {},
    "priority": "high",
    "severity": "critical",
    "status": "open"
  }
}
```

PATCH /issues/:id**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Issue updated successfully",
  "data": {
    "_id": "string",
    "issueId": "ISS101",
    "status": "in-progress",
    "updatedAt": "ISO_DATE"
  }
}
```

DELETE /issues/:id**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Issue deleted successfully"
}
```

7. Comment APIs**POST /comments****Status Code**

201

Expected Response

```
{
  "success": true,
  "message": "Comment added successfully",
  "data": {
    "_id": "string",
    "commentId": "COM101",
    "message": "Issue reproduced successfully",
    "createdAt": "ISO_DATE"
  }
}
```

GET /comments**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Comments fetched successfully",
  "data": [
    {
      "_id": "string",
      "commentId": "COM101",
      "message": "Issue reproduced successfully"
    }
  ]
}
```

```
}
```

GET /comments/:id**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Comment fetched successfully",
  "data": {
    "_id": "string",
    "commentId": "COM101",
    "message": "Issue reproduced successfully",
    "user": {},
    "issue": {}
  }
}
```

DELETE /comments/:id**Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Comment deleted successfully"
}
```

8. Workflow APIs**PATCH /issues/:id/assign****Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Issue assigned successfully",
  "data": {
    "issueId": "ISS101",
    "assignedTo": {
```

```
{
  "_id": "string",
  "name": "Rahul"
}
```

PATCH /issues/:id/status

Allowed Status

open

in-progress

testing

resolved

closed

Status Code

200

Expected Response

```
{
  "success": true,
  "message": "Issue status updated successfully",
  "data": {
    "issueId": "ISS101",
    "status": "resolved",
    "updatedAt": "ISO_DATE"
  }
}
```

Workflow Validation Rules

- closed issues cannot move back without reopen
- resolved issues cannot be edited directly
- testers cannot close issues directly
- only assigned developer can move issue to testing

9. Filtering, Search & Pagination APIs

GET /issues?priority=high

GET /issues?status=open

GET /issues?severity=critical

Status Code

200

Expected Response

```
{
  "success": true,
  "message": "Issues filtered successfully",
  "data": [
    {
      "_id": "string",
      "issueId": "ISS101",
      "priority": "high"
    }
  ]
}
```

GET /projects?status=active**GET /projects?owner=Rahul****Status Code**

200

Expected Response

```
{
  "success": true,
  "message": "Projects filtered successfully",
  "data": [
    {
      "_id": "string",
      "projectId": "PROJ1001",
      "status": "active"
    }
  ]
}
```

GET /issues?page=1&limit=10**GET /issues?search=login****GET /issues?status=open&priority=high&page=2&limit=5****Status Code**

200

Expected Pagination Response

```
{
  "success": true,
  "message": "Issues fetched successfully",
  "page": 2,
```

```
"limit": 5,  
"total": 50,  
"totalPages": 10,  
"data": [  
  {  
    "_id": "string",  
    "issueId": "ISS101",  
    "title": "Login issue"  
  }  
]  
}
```

10. Analytics APIs

GET /analytics/issues

Status Code

200

Expected Response

```
{  
  "success": true,  
  "message": "Issue analytics fetched successfully",  
  "data": {  
    "totalIssues": 100,  
    "openIssues": 35,  
    "resolvedIssues": 40,  
    "closedIssues": 20  
  }  
}
```

Values are dynamic.

GET /analytics/projects

Status Code

200

Expected Response

```
{  
  "success": true,  
  "message": "Project analytics fetched successfully",  
  "data": [  
    {  
      "project": "Bug Tracker",  
      "issueCount": 42,  
    }  
  ]  
}
```

```
    "status": "active"
  }
]
}
```

Values are dynamic.

GET /analytics/developers

Status Code

200

Expected Response

```
{
  "success": true,
  "message": "Developer analytics fetched successfully",
  "data": [
    {
      "developer": "Rahul",
      "resolvedIssues": 15,
      "averageResolutionTime": 4,
      "highestResolvedIssueCount": 15
    }
  ]
}
```

Values are dynamic.

11. Frontend Evaluator Contract

Mandatory React Router Routes

Route	Purpose
/login	Authentication
/dashboard	Analytics dashboard
/users	User management
/projects	Project management
/issues	Issue management
/comments	Comment management
/profile	Current user profile

Mandatory window.appState Structure

Students must expose:

window.appState

Expected Structure

```
window.appState = {  
  authUser,  
  token,  
  users,  
  projects,  
  issues,  
  comments,  
  filters,  
  analytics  
}
```

Runtime State Validation

The evaluator may validate state updates dynamically after:

- POST operations
- PUT operations
- PATCH operations
- DELETE operations
- login/logout actions
- filtering/search actions
- pagination actions